ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ

ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«Колледж автоматизации и информационных технологий № 20»

КУРСОВОЙ ПРОЕКТ

Тема Проектирование и разработка приложения для учета тренировок и достижений в спортивном клубе
(наименование темы)

Код и наименование специальности 09.02.07 Информационные системы и программирование
Группа ИСП222

Студент _____/ Кутас Владислав Александрович
(подпись) (фио)

Руководитель _____/Лисавина Алёна Вадимовна
(подпись) (фио)

Москва, 2026

ДЕПАРТАМЕНТ ОБРАЗОВАНИЯ И НАУКИ ГОРОДА МОСКВЫ

ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ПРОФЕССИОНАЛЬНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
«Колледж автоматизации и информационных технологий № 20»

РАССМОТРЕНО:
на заседании ППО
«Информационные технологии»
от «09» февраля 2026 г.
Протокол № 4
Председатель ППО
_____/Кузьмина О.Б.
(подпись) (Ф.И.О.)

Руководитель по курсовому проекту
_____/ Лисавина А.В.
(подпись) (Ф.И.О.)

УТВЕРЖДАЮ:
Заместитель директора по качеству образования
_____/ Глазков П.А.
(подпись) (Ф.И.О.)
«___»_____ 2026 г.

ЗАДАНИЕ
на курсовой проект

Студенту(ке) Кутас Владиславу Александровичу
(Ф.И.О. полностью)

Тема Проектирование и разработка приложения для учета тренировок и достижений в спортивном клубе
(наименование темы)

Содержание курсового проекта	**Сроки исполнения задания**	**Примечание**
ВВЕДЕНИЕ (~ 5%)	11.03.2026-16.03.2026	
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР ТЕХНОЛОГИЧЕСКОГО СТЕКА (~ 25-35 %) 1.1 Анализ предметной области 1.1.1 Описание бизнес-процессов 1.1.2 Выявление проблем и обоснование необходимости автоматизации 1.1.3 Анализ существующих решений	17.03.2026-23.03.2026	
1.2 Формулировка требований к системе 1.2.1 Функциональные требования 1.2.2 Нефункциональные требования	24.03.2026-30.03.2026	
1.3 Выбор и обоснование стека технологий	31.03.2026	
2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ (~ 40-50 %) 2.1 Проектирование 2.1.1 Проектирование базы данных 2.1.2 Диаграмма вариантов использования	01.04.2026-08.04.2026	
2.2 Разработка 2.2.1 Реализация модели базы данных 2.2.2 Разработка приложения 2.2.2.1 Реализация клиентской части	09.04.2026-20.04.2026	

2.2.2.2 Разработка серверной части		
2.3 Организация хранения и доступа к файлам посредством репозитория	21.04.2026	
2.4 Тестирование системы	22.04.2026	
ЗАКЛЮЧЕНИЕ (~ 8-10 %)	23.04.2026	
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	24.04.2026	

Дата выдачи задания «09» февраля 2026 г.

Задание получил _____ «09» февраля 2026 г.

(подпись студента)

СОДЕРЖАНИЕ

СПИСОК СОКРАЩЕНИ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ 6

ВВЕДЕНИЕ..... 7

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР ОПТИМАЛЬНОГО ТЕХНОЛОГИЧЕСКОГО СТЕКА 9

1.1 Анализ предметной области 9

1.1.1 Описание бизнес-процессов..... 9

1.1.2 Выявление проблем и обоснование необходимости автоматизации 10

1.1.3 Анализ существующих решений..... 11

1.2 Формулировка требований к системе 13

1.2.1 Функциональные требования..... 13

1.2.2 Нефункциональные требования 14

1.3 Выбор и обоснование стека технологий..... 14

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ 18

2.1 Проектирование..... 18

2.1.1 Проектирование базы данных..... 18

2.1.2 Диаграмма вариантов использования 20

2.2 Разработка 22

2.2.1 Реализация модели базы данных 22

2.2.2 Разработка приложения 25

2.3 Организация хранения и доступа к файлам посредством репозитория 33

2.4 Тестирование системы..... 34

ЗАКЛЮЧЕНИЕ 38

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 40

ПРИЛОЖЕНИЯ 43

ПРИЛОЖЕНИЕ А. Программный код страницы «Авторизация»..... 43

ПРИЛОЖЕНИЕ Б. Программный код страницы «Регистрация» 45

ПРИЛОЖЕНИЕ В. Программный код страницы «Расписание тренировок». 47

ПРИЛОЖЕНИЕ Г. Программный код страницы «Редактирование тренировок» 50

ПРИЛОЖЕНИЕ Д. Программный код страницы «Достижения» 54
ПРИЛОЖЕНИЕ Е. Программный код страницы «Редактирование достижений» 56
ПРИЛОЖЕНИЕ Ж. Скрипт базы данных проекта 58

СПИСОК СОКРАЩЕНИ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

CRM (от англ. «Customer Relationship Management») – это функциональный инструмент, который помогает выстраивать и поддерживать отношения с клиентами.

IDE, (от англ. Integrated Development Environment, «интегрированная среда разработки») – интегрированная среда разработки, набор ПО для создания кода.

ER (Entity-Relationship, или диаграмма «сущность-связь») – это схема, которая показывает, какие данные есть в системе и как они связаны друг с другом.

SQL-инъекция – техника, при которой злоумышленник пользуется недостатками в коде приложения, который отвечает за построение динамических SQL-запросов.

Git Bash – это приложение для сред Microsoft Windows, эмулирующее работу командной строки Git.

Git – это система для управления версиями исходного кода программ.

WPF – это платформа для разработки Windows-приложений с современными пользовательскими интерфейсами.

ВВЕДЕНИЕ

В современном мире здоровье и красота важный фактор успешности человека. Расширение сферы услуг в области спорта подтолкнуло многие организации на создание специализированных приложений спортивных клубов. В новом технологичном мире потребителю намного проще записываться на тренировки онлайн к любимому тренеру, а не тратить время на бумажные расписания и очереди [1, с.37-41].

Применение специализированной информационной системы позволяет упростить работу сотрудников. Администраторы получают больше возможностей для гибкого управления расписанием тренировок, контроля записи клиентов.

Существующие универсальные системы управления бизнесом не могут удовлетворять все задачи деятельности спортивного клуба. Такие системы имеют много лишнего функционала, сложны в освоении и требуют привлечения сторонних специалистов для дополнительной настройки. Из-за этого возникает необходимость разработки специализированного приложения, ориентированного под конкретные задачи и имеющего низкий порог входа для пользователей.

Целью проекта является проектирование и разработка приложения учета тренировок и достижений в спортивном клубе. Приложение позволит автоматизировать процессы регистрации тренировок, их изменения, а также записи достижений клуба.

Задачи проекта:

1) Анализ предметной области и существующих решений;

2) Формулировка функциональных и нефункциональных требований к системе;

3) Выбор оптимального технологического стека для реализации приложения;

4) Проектирование и разработка системы;

5) Тестирование.

Объектом исследования является деятельность спортивных клубов, связанная с организацией тренировок и достижений.

Предметная область проекта связана с деятельностью спортивных клубов. Спортивные клубы представляют собой организацию, которая оказывает услуги в области физической подготовки и развития, реализуя свою деятельность они формируют расписание тренировок, распределяет клиентов по занятиям и ведет учет проведенных тренировок. Предметная область спортивных клубов охватывает процессы организации и учета тренировок, а также фиксирует его достижения.

Значимость данного проекта заключается в разработке приложения, которое будет использовано спортивными организациями для автоматизации всех процессов.

Применение данного приложения позволит повысить эффективность работы спортивного клуба и сократить время на обработку данных, снизить вероятность возникновения ошибок и улучшить взаимодействие между тренером и клиентом.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ВЫБОР ОПТИМАЛЬНОГО ТЕХНОЛОГИЧЕСКОГО СТЕКА

1.1 Анализ предметной области

1.1.1 Описание бизнес-процессов

Спортивные клубы ведут несколько ключевых бизнес-процессов, которые обеспечивают функционирование организации. К ним относятся:

— Регистрация клиентов;

— Планирование тренировок;

— Отслеживание достижений клуба.

При регистрации клиента создается его учетная запись в системе клуба. Клиент предоставляет свои данные, такие как имя, логин, пароль и роль. Когда клиент зарегистрировался все предоставленные им данные записываются в базу данных системы для дальнейшего использования в записи тренировок и авторизации с использованием логина и пароля.

Тренер имеет возможность создавать расписание с указанием всех необходимых данных, таких как время начала и конца тренировки, дата занятия, вид спорта и ответственного за тренировку тренера. В это время клиенты получают возможность просмотра своего расписания и записи на занятия с интересующим их тренером. В случае если клиент передумал или хочет изменить дату занятия тренер всегда может перенести тренировку.

Таким же важным бизнес-процессом является отслеживание достижений спортивного клуба. Тренер, в роли администратора, может добавлять заслуги клуба, а клиенты в любой момент могут просмотреть насколько престижный спортивный клуб и чем он выделяется от остальных.

1.1.2 Выявление проблем и обоснование необходимости автоматизации

На данный момент многие клубы используют бумажные журналы, Excel-таблицы или стандартные CRM-системы[1], такие как 1С: Предприятие или Bitrix24.

Основные проблемы существующих решений:

— Риск потери данных;

— Низкая скорость обработки информации;

— Отсутствие специализированного функционала.

При ручном ведении учета клиентов существует высокая вероятность потери информации, а также понижения скорости обработки данной информации из-за человеческого фактора. Все это снижает общую эффективность работы клуба.

Во многих вышесказанных системах учета нет ориентирования на специфику работы спортивных клубов из-за чего администраторам приходится адаптировать инструменты под конкретные задачи, это неудобно и неэффективно.

Автоматизация данных процессов через разработку специализированного приложения позволит:

— Ускорить учет тренировок;

— Снизить вероятность ошибок;

— Вести статистику достижений клуба;

— Повысить уровень удовлетворенности клиентов.

Таким образом можно сделать вывод что разработка специализированного настольного приложения является актуальной в современных реалиях.

[1] CRM (от англ. «Custome Relationship Management») – это функциональный инструмент, который помогает выстраивать и поддерживать отношения с клиентами. [Электронный ресурс] – URL: https://crm.ru/blog/crm-chto-eto-takoe Режим доступа: свободный (дата обращения: 20.03.2026).

1.1.3 Анализ существующих решений

Для наглядного сравнения рассмотренных аналогов представим их основные характеристики в виде таблицы 1.

Таблица 1 – Сравнение ИС

Название	1С: Предприятие	Bitrix24
Стоимость	Высокая	Средняя
Сложность освоения	Высокая	Высокая
Гибкость настройки	Низкая	Средняя

Также на рисунке 1 и рисунке 2 представлены интерфейсы данных платформ.

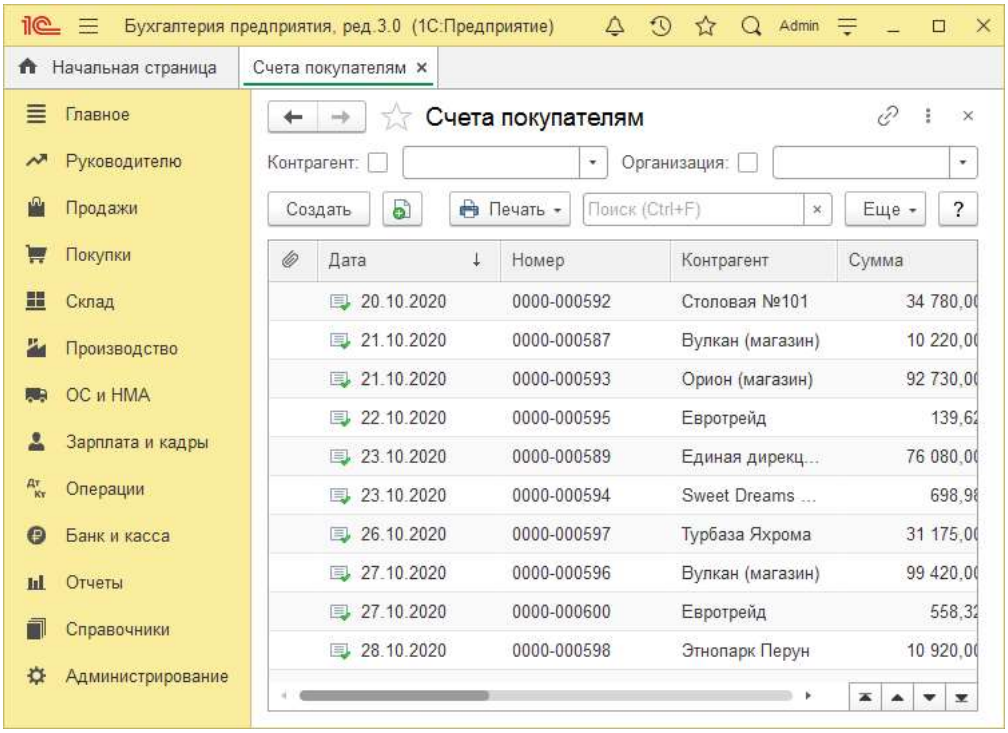

Рисунок 1 – Интерфейс платформы 1С: Предприятие

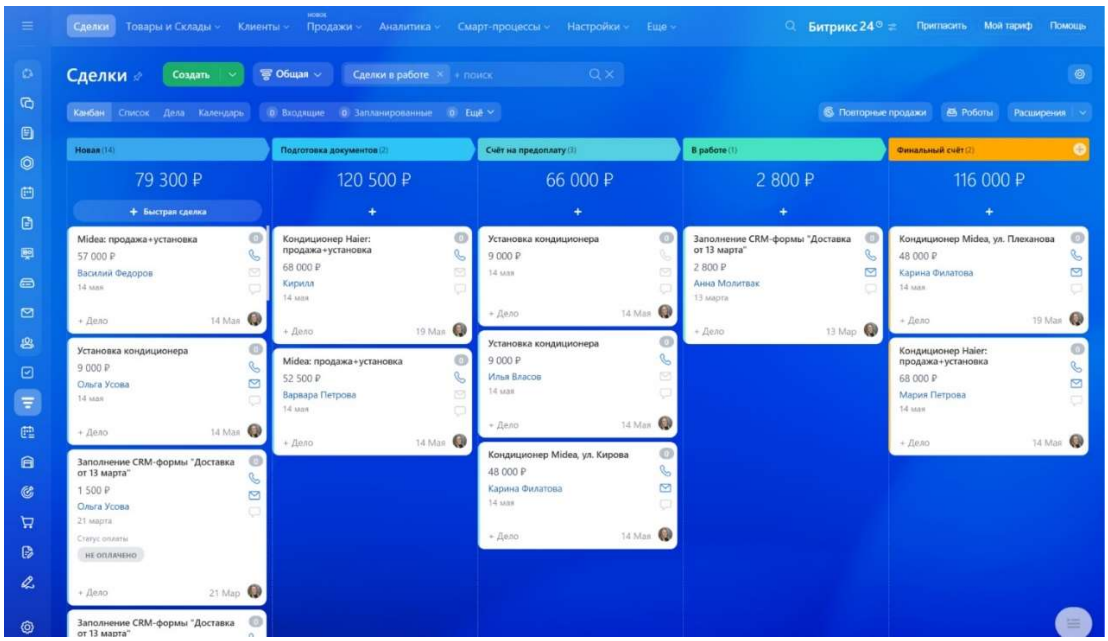

Рисунок 2 – Интерфейс приложения Bitrix24

Стоимость внедрения и использования программного обеспечения важный фактор при выборе информационной системы. Платформа 1С: Предприятие имеет высокую стоимость из-за требования к приобретению лицензии, оплаты дополнительных модулей и услуг специалистов по настройке и внедрению системы [18]. Для работы с данной платформы необходимо обучать сотрудников работе с ней. Система Bitrix24 предлагает различные тарифы, но имеет бесплатную версию с ограниченном функционалом. Платные варианты имеют среднюю стоимость по сравнению с аналогами, что делает данный продукт более привлекательным и доступным для малого и среднего бизнеса.

Еще одним важным параметром является сложность освоения платформы. Система 1С: Предприятие имеет сложный порог вхождение и для пользования нужны специалисты с соответствующей квалификацией. В отличии от 1С: Предприятие, Bitrix24 имеет более современный интерфейс, но из-за большого количества функций и инструментов система может вызвать трудности у новых пользователей. Для обучения сотрудников потребуется достаточно много времени что не выгодно для работодателя.

Гибкость настройки подразумевает под собой возможность настройки системы для решения конкретных задач организации. Платформа 1С: Предприятие имеет широкие возможности настройки, но для изменений требуется

помощь программиста, а это усложняет процесс настройки системы. Система Bitrix24 предоставляет большой список встроенных инструментов для настройки без участия IT специалиста, но уровень гибкости малый.

Эти системы популярны для общего пользования, но они не ориентированы на специфику деятельности спортивных клубов. В них нет специализированных инструментов для удобного планирования тренировок, управления расписанием занятий и учета спортивных достижений.

Пользование этими системами требует дополнительной настройки с привлечением сторонних специалистов, что невыгодно организации. Поэтому разработка программного продукта, ориентированного на задачи спортивного клуба будет самым эффективным подходом.

1.2 Формулировка требований к системе

На основе проведенного анализа предметной области необходимо определить требования к разрабатываемому программному продукту. Четкое формулирование требований снижает вероятность ошибок в разработке и обеспечивает соответствие разрабатываемой информационной системы поставленным целям.

1.2.1 Функциональные требования

Функциональные требования определяют основные возможности приложения и его взаимодействия с пользователями. Реализация данных функций позволит автоматизировать все процессы спортивного клуба и в результате он будет работать быстрее и улучшит обслуживание клиентов.

Разрабатываемая система должна обеспечивать:

— Регистрацию и авторизацию клиентов, управление их данными;
— Изменение расписания администратором;
— Учет достижений клуба по видам спорта;
— Просмотр и назначение тренировок;
— Удобный интерфейс.

1.2.2 Нефункциональные требования

Нефункциональные требования описывают основные качества системы, ее безопасность и стабильную, бесперебойную работу. Соблюдение этих требований обеспечит быструю обработку запросов, безопасность информации и возможности для развития системы [4, с. 17-28]. Это важно для приложений работающих с большим количеством пользователей и данных. Нефункциональные требования разрабатываемой системы:

1) Высокая производительность;
2) Надежность и безопасность хранения данных;
3) Масштабируемость системы для добавления новых функций.

1.3 Выбор и обоснование стека технологий

Для выбора языка программирования были рассмотрены самые распространенные варианты, которые применяются при разработке программных решений. Результаты сравнения представлены на таблице 2.

Таблица 2 – Сравнение языков программирования

Параметр	C#	Java	Python
Производительность	Высокая	Высокая	Средняя
Простота изучения	Средняя	Средняя	Высокая
Популярность	Высокая	Высокая	Высокая

Из таблицы 2 видно, что язык программирования C# и Java выделяются высокой производительностью и популярностью, но C# подходит лучше для разработки приложения из-за удобной интеграции с инструментами Microsoft и поддержке современных технологий разработки [13]. Python более простой в структуре и изучении, но по производительности он менее предпочтителен чем C# и Java.

Для выбора среды разработки были проанализированы популярные программные средства, которые используются для создания приложений на платформе .NET. Результаты представлены на таблице 3.

Таблица 3 – Сравнение Integrated Development Environment (IDE)[1]

Параметр	Visual Studio	Visual Studio Code	Rider (JetBrains)
Полнота функций	Высокая	Средняя	Высокая
Интеграция с .NET	Отличная	Хорошая	Отличная
Стоимость	Бесплатная	Бесплатная	Платная
Производитель-ность	Высокая	Высокая	Высокая

Visual Studio и Rider имеют высокую функциональность и интеграцию с .NET, но Visual Studio выделяется за счет бесплатной версии. Visual Studio Code бесплатный и легковесный, но по функциональности и глубине интеграции с платформой .NET уступает другим представленным аналогам. Rider ограничен своей платной лицензией, именно это снижает его доступность.

При разработке настольных и веб-приложений на платформе .NET существует несколько технологий для создания пользовательского интерфейса. Наиболее распространенными являются:

— Windows Forms;

— WPF;

— ASP.NET.

Каждая из этих технологий имеет свои особенности и преимущества. Для выбора наиболее подходящей нужно рассмотреть их характеристики в таблице 4.

Таблица 4 – Сравнение технологий создания пользовательского интерфейса

Параметр	Windows Forms	WPF	ASP.NET
Тип приложения	Настольное	Настольное	Веб-приложение
Сложность освоения	Низкая	Средняя	Высокая
Гибкость интерфейса	Средняя	Высокая	Высокая
Производительность	Высокая	Высокая	Зависит от сер-вера
Требования к интернету	Нет	Нет	Да

Windows Forms одна из наиболее простых и доступных для разработки настольных приложений. Она обладает низких порогом вхождения и позво-

[1] IDE (от англ. Integrated Development Environment, «интегрированная среда разработки») — интегрированная среда разработки, набор ПО для создания кода. [Электронный ресурс] – URL: https://blog.skillfactory.ru/glossary/ide/ Режим доступа: свободный (дата обращения: 30.03.2026).

ляет быстро создавать графические интерфейсы. Из-за простоты использования и большому количеству готовых компонентов эта технология широко применяется для разработки информационных систем [13].

Windows Presentation Foundation представляет собой более современную технологию, предоставляющую расширенные возможности создания сложных элементов и гибкую настройку пользовательского интерфейса. Но использование данной технологии требует более глубоких знаний, что увеличивает сложность разработки.

ASP.NET используется для разработки веб-приложений, работающих через браузер. Данная технология позволяет создавать масштабируемые и гибкие системы, но для работы таких приложений требуется веб-сервер и постоянное подключение к сети.

Для выбора системы управления базой данных были рассмотрены наиболее известные аналоги, которые используются при разработке информационных систем. Результат представлен на таблице 5.

Таблица 5 – Сравнение систем управления базами данных

Параметр	MySQL	PostgreSQL	Oracle
Стоимость	Бесплатная	Бесплатная	Высокая
Производительность	Высокая	Высокая	Очень высокая
Сложность настройки	Низкая	Средняя	Высокая
Масштабируемость	Средняя	Высокая	Очень высокая
Популярность	Очень высокая	Высокая	Высокая

MySQL и PostgreSQL выгодны за счет своей бесплатности и высокой производительности, но MySQL имеет низкую сложность настройки и популярность [14]. PostgreSQL превосходит аналоги по масштабируемости, но уступает в аспектах администрирования. Oracle имеет отличную производительность и масштабируемость, но ограничен стоимостью и сложностью внедрения.

Для создания визуального построения диаграмм используется программа Microsoft Visio.

Данная программа широко применяется для проектирования информационных систем и наглядно отображает структуру программного обеспечения, бизнес-процессы и архитектуру системы.

Основные преимущества программы:

— Удобный интерфейс;

— Широкий выбор шаблонов и элементов;

— Возможность создания большого количества типов диаграмм;

— Интеграция с продуктами Microsoft Office.

Проанализировав все варианты инструментов для разработки проекта, был выбран следующий стек технологий:

— C#

— Visual Studio

— Windows Forms

— MySQL

— Visio

Выбранный стек технологий полностью соответствует требованиям проекта, не создавая дополнительной нагрузки и ускоряет разработку функционала.

2 ПРОЕКТИРОВАНИЕ И РАЗРАБОТКА СИСТЕМЫ

2.1 Проектирование

Во время проектирования можно понять, как будет выглядеть система, какие функции она будет выполнять и как пользователь будет с ней взаимодействовать. От успешности выполнения данного шага зависит качество всей разработки. Ошибки в ходе проектирования сложно исправить на поздних этапах разработки [2, с.16-29].

При разработке системы особое внимание уделялось удобству, так как данное приложение могут использовать не только технические специалисты, но и обычные сотрудники и клиенты клуба. Поэтому интерфейс должен быть интуитивно понятным, а логика работы максимально простой.

Было решено разделить права доступа между ролями. Это разделение логично для спортивного клуба, где тренер напрямую управляет процессом тренировок, а клиент является его участником. Клиент получает доступ к просмотру собственных тренировок, а тренер может работать со всеми записями. Это не только повышает безопасность системы, но и делает ее использование более логичным и понятным.

Логика работы системы построена таким образом, что пользователь проходит определенные этапы:

— Запуск приложения;
— Регистрация при необходимости;
— Авторизация;
— Переход к функционалу.

Когда пользователь прошел данные этапы, система автоматически определяет роль и открывает соответствующие формы и кнопки, это помогает разграничивать доступ к необходимым данным.

2.1.1 Проектирование базы данных

База данных нужна для хранения информации и ее доступности в проекте. При проектировании базы данных было важно правильно структурировать данные и предотвратить ошибки [14].

Чтобы данные не повторялись, база данных сделана по принципам нормализации [5, с. 181-190]. Отдельные таблицы для:

— Пользователей;

— Видов спорта;

— Тренировок;

— Достижений.

Такое разделение помогает разграничить данные и упростить работу с ними. На рисунке 3 представлена ER-диаграмма[1].

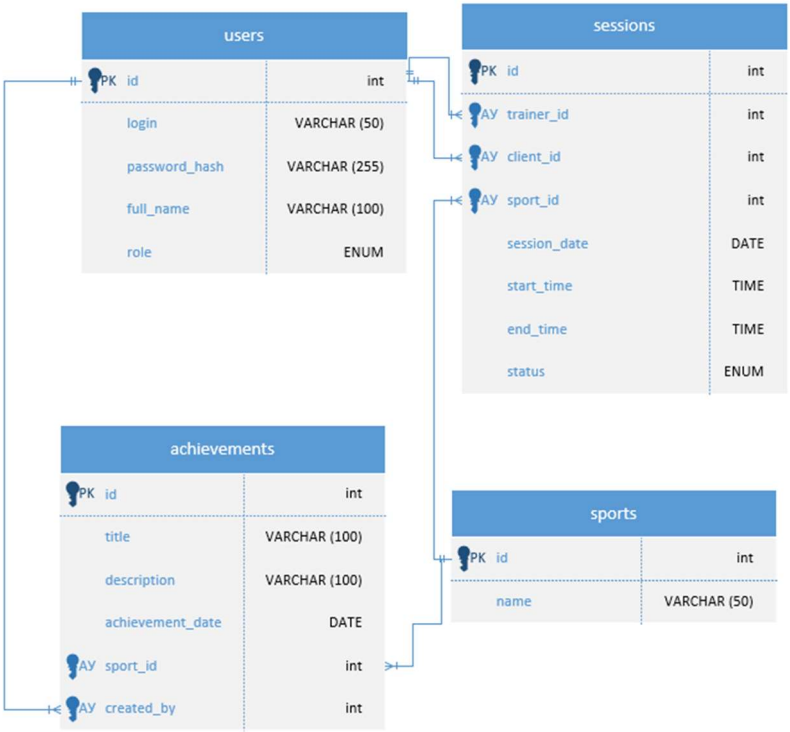

Рисунок 3 – ER-диаграмма

Таблица пользователей содержит информацию о зарегистрированных в системе пользователях. Было сделано так, что каждый пользователь имеет свой уникальный логин для авторизации. В таблице хранится роль пользователя для разграничения доступа. Эти ограничения обеспечивают корректность данных и снижают риск ошибок.

[1] ER (Entity-Relationship, или диаграмма «сущность-связь») – это схема, которая показывает, какие данные есть в системе и как они связаны друг с другом. [Электронный ресурс] – URL: https://kurshub.ru/journal/blog/chto-takoe-er-diagramma/ Режим доступа: свободный (дата обращения: 05.04.2026).

Таблица видов спорта является вспомогательной таблицей и используется в качестве справочника для централизованного хранения информации о доступных видах спорта. Это упрощает работу с данными и делает систему удобной для использования.

Таблица тренировок самая важная из всех таблиц в системе, потому что в ней хранится основная информация о занятиях. При проектировании учитывались реальные условия при которых один тренер не может проводить несколько тренировок в одно и тоже время. Реализована проверка корректности времени. Она не дает указать время окончания тренировки раньше времени начала.

Таблица достижений используется для хранения информации о достижениях. Отдельная таблица, в которую записываются только достижения самой организации, без учета пользователей. Это упрощает структуру самой базы данных.

Связи между таблицами реализованы при помощи внешних ключей для обеспечения целостности данных. Система не позволит создать запись, которая ссылается на несуществующие данные. Этот механизм позволяет обеспечить надежность системы [15].

В дополнение были реализованы индексы, которые позволяют ускорить выполнение запросов к базе данных. Это важно для работы с таблицей тренировок, потому что она используется чаще всех остальных таблиц и имеет наибольшее количество записей.

2.1.2 Диаграмма вариантов использования

Диаграмма вариантов использования показывает, как пользователи взаимодействуют с системой. На рисунке 3 представлена диаграмма вариантов использования для разрабатываемого приложения.

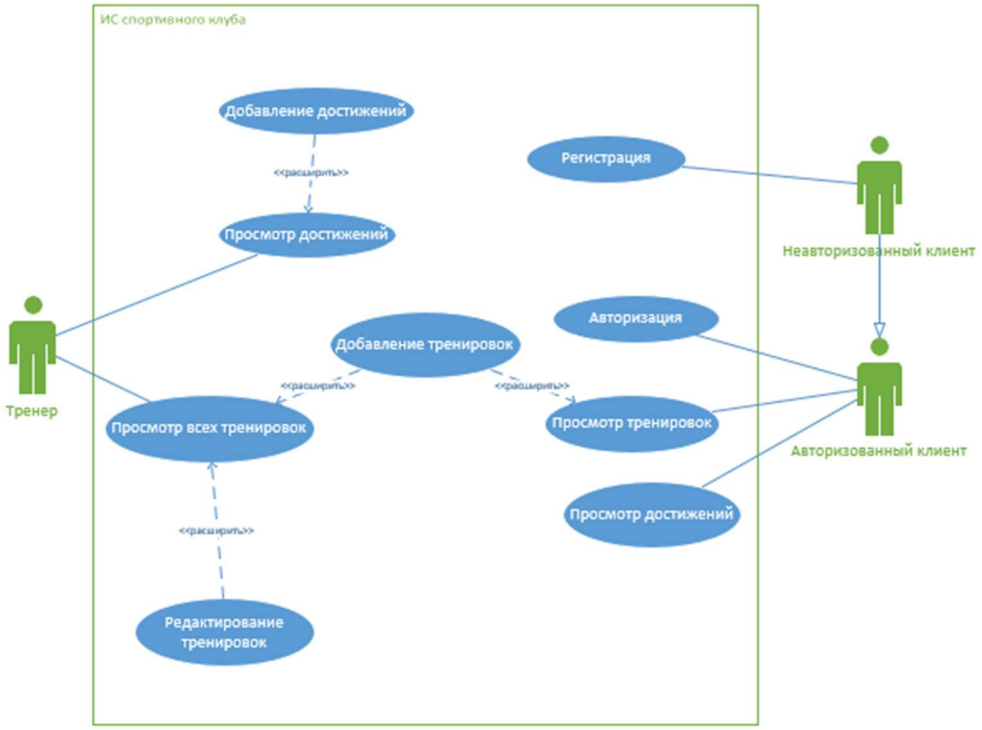

Рисунок 4 – Диаграмма вариантов использования

В данной информационной системе имеется два основных типа пользователей:

— Клиент;

— Тренер.

Каждый из них выполняет определенный набор действий, соответствующий его роли.

Клиент взаимодействует с системой на базовом уровне. Он проходит регистрацию, выполняет вход в систему и получает доступ к просмотру своих тренировок. Клиент не может менять данные и это исключает вероятность ошибок.

Тренер обладает расширенным функционалом. Кроме просмотра информации, как клиент, он может добавлять новые тренировки, редактировать существующие данные о них и работать с достижениями.

Благодаря разделению ролей, пользоваться системой стало удобнее и безопасней.

2.2 Разработка

На этапе разработки все решения проектирования превращаются в реальную, рабочую систему. На прошлом этапе создавалась структура системы и определялась ее логика, а в этом производится создание базы данных, программного кода и пользовательского интерфейса [17].

Разработка системы проводилась поэтапно. Это позволяет обеспечить последовательную реализацию всех компонентов и упростить процесс отладки. Сначала создавалась модель базы данных, потом разрабатывалась клиентская часть приложения и в конце выполнялась настройка взаимодействий с базой данных. Тако способ помог уменьшить количество ошибок и сделал работу системы надежной

2.2.1 Реализация модели базы данных

После проектирования была выполнена реализация базы данных. По сути, это практическое воплощение разработанной структуры. Основные действия для реализации базы данных:

— Создание таблиц;

— Настройка полей;

— Определение типов данных и ограничений.

Создание базы данных происходит с использованием SQL-запросов. Сначала была создана сама база данных, потом в ней были формированы таблицы. Такой способ снизил риск ошибок и позволил контролировать каждый шаг работы.

Тщательно подбирались типы данных. Они влияют на хранение сведений и стабильность работы системы. Например, для хранения дат использовались специальные типы данных, упрощающие сортировку и сравнение значений [15]. На рисунке 3 показан процесс создания структуры базы данных.

```sql
1  CREATE DATABASE IF NOT EXISTS sport_club;
2  USE sport_club;
3
4  CREATE TABLE users (
5      id INT AUTO_INCREMENT PRIMARY KEY,
6      login VARCHAR(50) NOT NULL UNIQUE,
7      password_hash VARCHAR(255) NOT NULL,
8      full_name VARCHAR(100) NOT NULL,
9      role ENUM('client','trainer') NOT NULL,
10     created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
11 ) ENGINE=InnoDB;
12
13 CREATE TABLE sports (
14     id INT AUTO_INCREMENT PRIMARY KEY,
15     name VARCHAR(50) NOT NULL UNIQUE
16 ) ENGINE=InnoDB;
17
18 CREATE TABLE sessions (
19     id INT AUTO_INCREMENT PRIMARY KEY,
20
```

Рисунок 5 – Создание структуры базы данных

После создания таблиц были добавлены ограничения. Они нужны для обеспечения целостности данных. Были реализованы первичные ключи, которые помогают однозначно идентифицировать каждую запись и внешние ключи, которые связывают таблицы между собой. Благодаря этому исключается появление несогласованных данных. SQL скрипт для создания ограничений представлен на рисунке 4.

```sql
    FOREIGN KEY (trainer_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE CASCADE,
    FOREIGN KEY (sport_id) REFERENCES sports(id) ON DELETE RESTRICT,
    FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE RESTRICT
) ENGINE=InnoDB;
```

Рисунок 6 – Определение ключей и связей между таблицами

Индексы ускоряют выполнение запросов при работе с большими объемами информации. Без них поиск может занимать больше времени. Они добавлены для часто используемых полей. Создание индексов представлено на рисунке 5.

```sql
CREATE UNIQUE INDEX ux_trainer_datetime
ON sessions (trainer_id, session_date, start_time);

CREATE INDEX ix_sessions_client ON sessions (client_id);
CREATE INDEX ix_sessions_date ON sessions (session_date);
```

Рисунок 7 – Создание индексов для оптимизации запросов

После реализации структуры, были добавлены тестовые данные для тестирования системы. После внесения тестовых данных стало проще разрабатывать клиентскую часть, потому что можно было сразу видеть результаты выполнения запросов. Код для добавления тестовых данных представлен на рисунке 6.

```sql
INSERT INTO users (login, password_hash, full_name, role)
VALUES
('client1', '123', 'Иван Иванов', 'client'),
('client2', '123', 'Алексей Смирнов', 'client'),
('trainer1', '123', 'Петр Петров', 'trainer');

INSERT INTO sports (name)
VALUES
('Фитнес'),
('Бокс'),
('Йога');

INSERT INTO sessions
(trainer_id, client_id, sport_id, session_date, start_time, end_time, status, created_by)
VALUES
(3, 1, 1, '2026-03-10', '10:00', '11:00', 'planned', 3),
(3, 2, 2, '2026-03-11', '12:00', '13:00', 'completed', 3),
(3, 1, 3, '2026-03-12', '14:00', '15:00', 'canceled', 3);

INSERT INTO achievements
(title, description, achievement_date, sport_id, created_by)
VALUES
('Лучший спортивный клуб года',
 'Победа в городском конкурсе среди фитнес-клубов',
 '2025-12-20', NULL, 3),
```

Рисунок 8 – Добавление тестовых данных в базу данных

Полный скрипт базы данных представлен в Приложении Ж.

Работа с базой данных в приложении выполняется с помощью SQL-запросов. Используются основные операции:

— Получение данных;

— Добавление записей;

— Изменение записей;

— Удаление записей.

Все запросы реализованы с параметрами. Это повышает безопасность и защищает систему от SQL-инъекций[1] [15].

[1] SQL-инъекция – техника, при которой злоумышленник пользуется недостатками в коде приложения, который отвечает за построение динамических SQL-запросов. [Электронный ресурс] – URL: https://www.php.net/manual/ru/security.database.sql-injection.php Режим доступа: свободный (дата обращения: 06.04.2026).

Все операции выполняются в конструкциях, которые перехватывают исключения. Это делает систему более устойчивой. В случае ошибки пользователь получает понятное сообщение, а не простой сбой программы.

Итоги реализации модели базы данных:

1) Система обеспечивает надежное хранение информации;
2) Система обеспечивает корректную обработку данных;
3) Система может быть использована в дальнейшем без серьезных доработок.

2.2.2 Разработка приложения

Разработка приложения самая объемная часть всей работы. Все принятые решения превращаются в программный код и пользовательский интерфейс.

Разработка делилась на этапы:

1) Создание форм и размещение элементов управления;
2) Реализация логики работы;
3) Подключение к базе данных.

Это упростило процесс разработки и сделало ее более понятной [3, с. 290-297].

Каждая форма отвечает за выполнение своей задачи. В приложении реализованы формы:

— Авторизации;
— Регистрации;
— Расписания тренировок;
— Редактирования данных;
— Просмотра достижений.

2.2.2.1 Реализация клиентской части

Клиентская часть представлена через набор форм, через которые пользователь взаимодействует с системой. Это важно, поэтому интерфейс должен быть понятным и удобным.

Работа начинается с формы авторизации. Это первая форма, которую видит пользователь. На ней расположены поля ввода логина и пароля и кнопки для входа и перехода к регистрации. При нажатии на кнопку входа выполняется проверка введенных данных. Потом происходит обращение к базе данных. Если данные корректные, то переход на основную форму если нет, то вывод сообщения об ошибке. При нажатии на кнопку «Регистрация» осуществляется переход на форму регистрации. Форма авторизации представлена на рисунке 9.

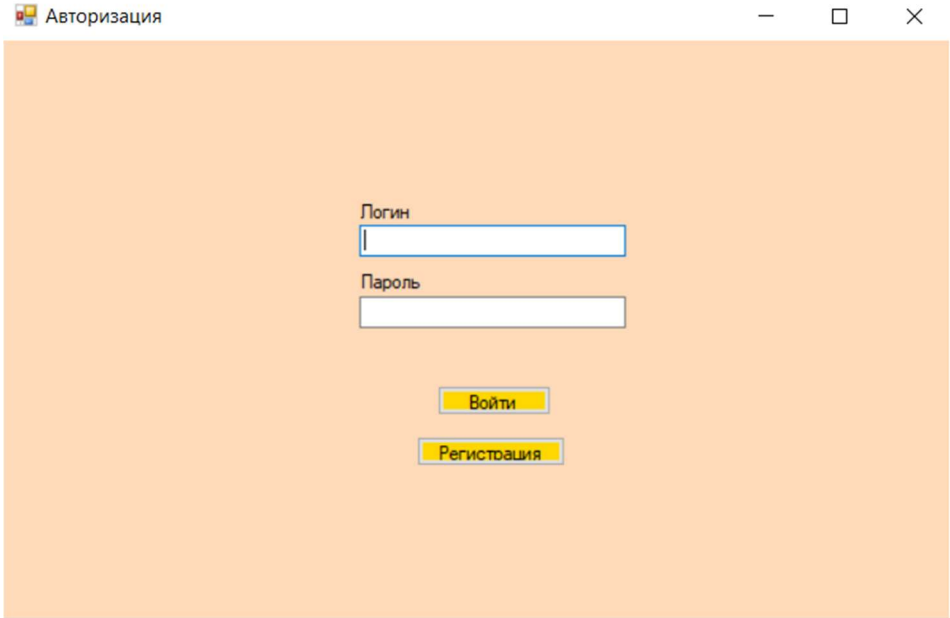

Рисунок 9 – Форма авторизации пользователя

Форма регистрации используется для создания новых пользователей. Здесь происходит проверка корректности введенных данных. Например, нельзя оставить поля пустыми, а логин должен быть уникальным. После успешной регистрации пользователь возвращается к форме авторизации. Форма регистрации представлена на рисунке 10.

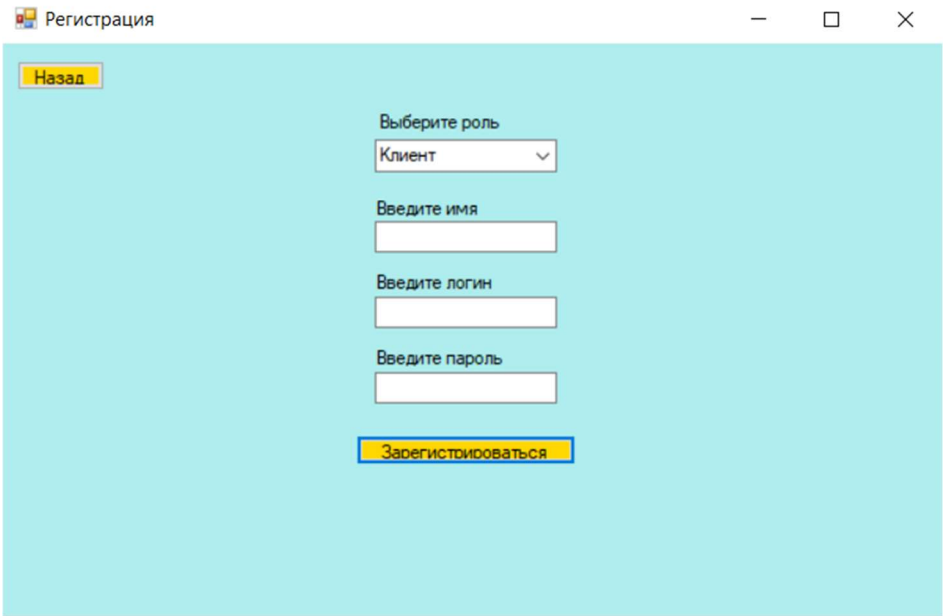

Рисунок 10 – Форма регистрации пользователя

Основной формой приложения является форма расписания тренировок. На ней отображаются данные из базы в виде таблицы. Используется элемент DataGridView, который удобен в представлении информации. Содержимое таблиц зависит от роли пользователя. Клиент видит только свои тренировки, а тренер видит все записи.

На форме расписания реализованы элементы управления в виде кнопок «Добавить», «Изменить», «Достижения». С их помощью можно использовать одноименные функции, такие как добавление тренировки, изменение тренировки и переход на форму с достижениями клуба. При нажатии кнопок открываются дополнительные формы. Это позволяет не перегружать основной интерфейс. Форма расписания тренировок для тренера и клиента представлена на рисунках 11 и 12.

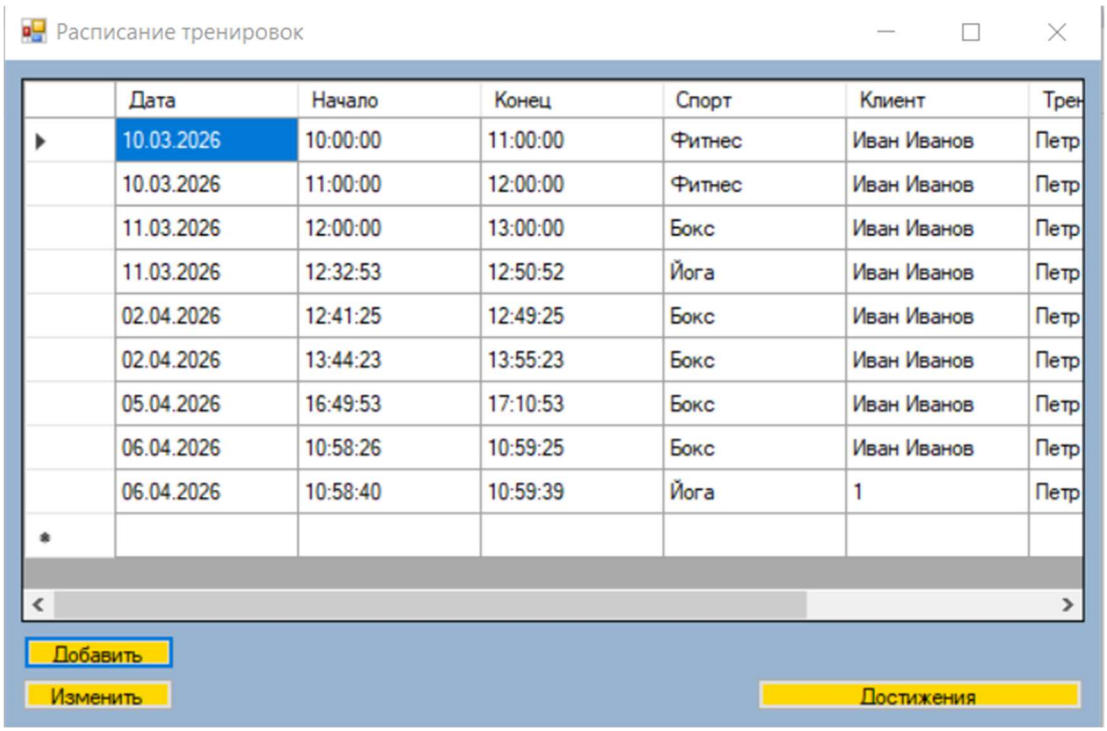

Рисунок 11 – Форма расписания тренировок для тренера

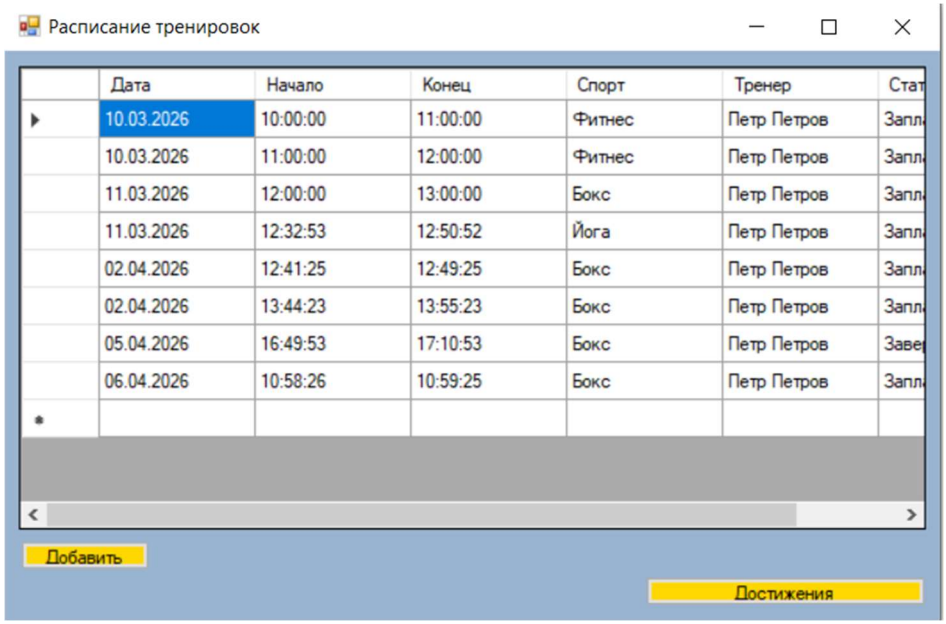

Рисунок 12 – Форма расписания для клиента

Форма добавления и редактирования тренировок универсальна. Она используется и для создания новых записей, и для изменения существующих. Пользователь задает параметры, такие как тренер, вид спорта, вид спорта, дата и время. После сохранения выполняется соответствующий SQL-запрос. Форма добавления и редактирования тренировок представлена на рисунке 13.

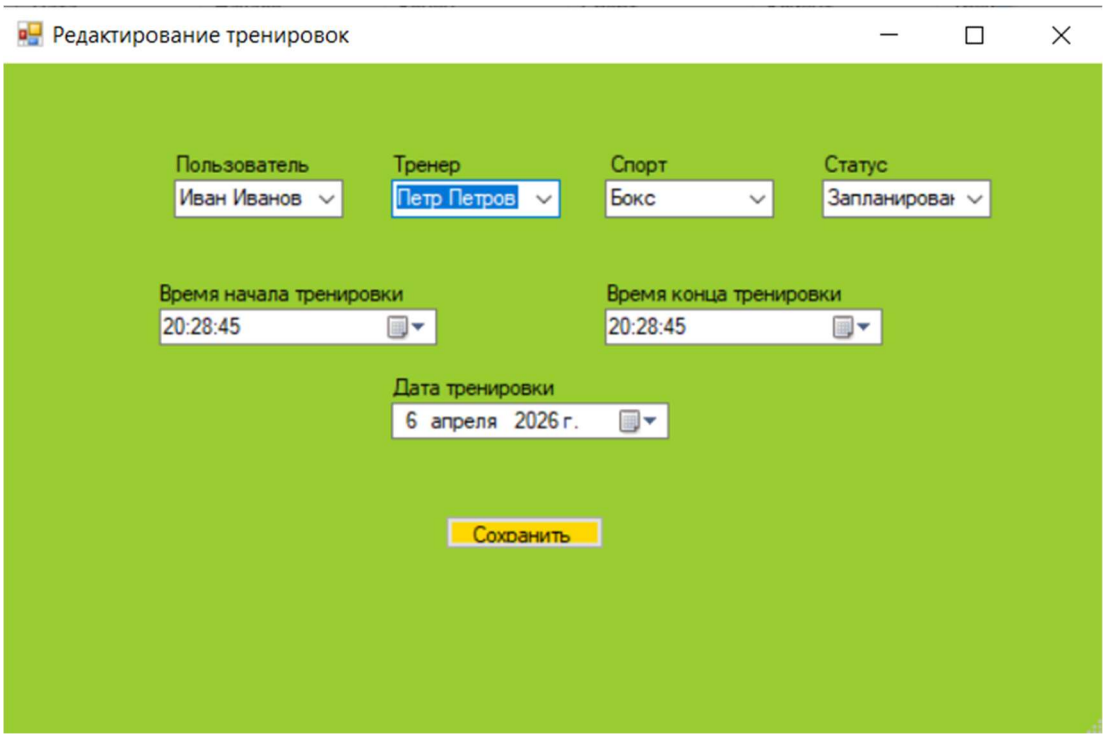

Рисунок 13 – Форма добавления и редактирования тренировки

Форма достижений нужна для отображения информации о достижениях клуба. Данные вводятся в табличном виде. Пользователь может просматривать информацию, а тренер имеет возможность добавлять новые записи. Форма достижений при входе тренера и клиента представлена на рисунках 14 и 15.

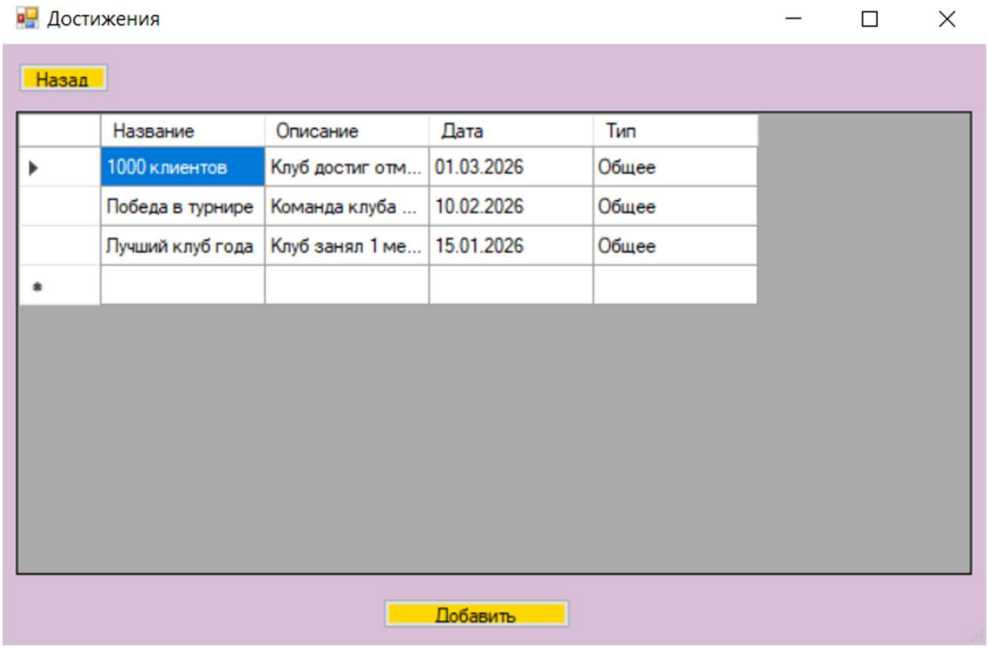

Рисунок 14 – Форма достижений клуба тренера

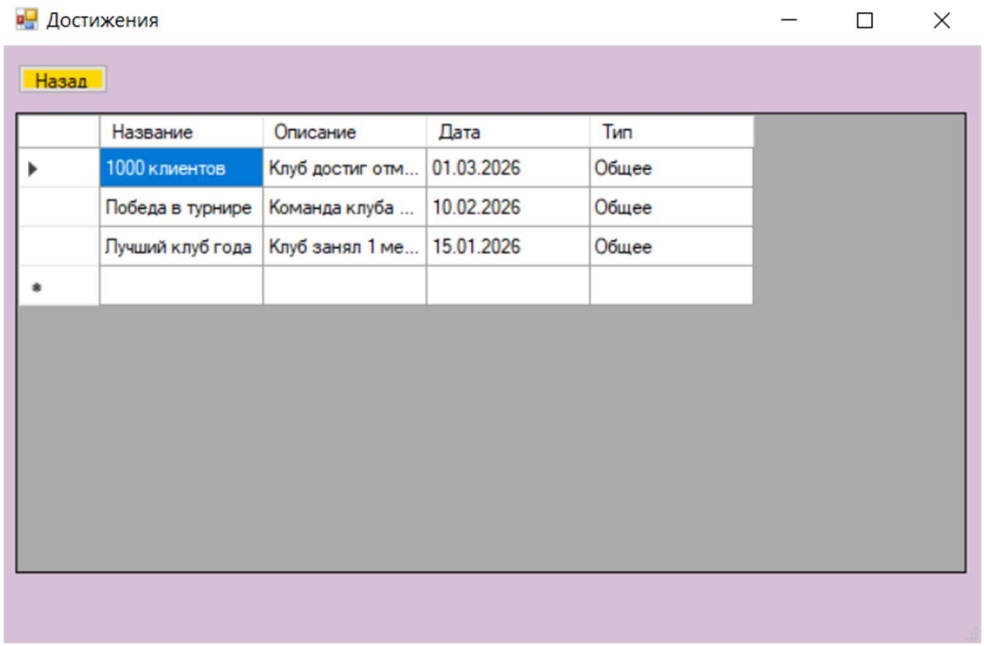

Рисунок 15 – Форма достижений для клиента

2.2.2.2 Разработка серверной части

Для взаимодействия с базой данных используется специально созданный класс, который отвечает за подключение и выполнение SQL-запросов. Это упрощает работу с приложением и централизует данные. Реализованный класс представлен на рисунке 16.

```
Ссылок: 12
internal class DbClass
{
    Ссылок: 11
    public static class Database
    {
        private static readonly string connectionString =
            "Server=localhost;Database=sport_club;Uid=root;Pwd=jokit007;";

        private static MySqlConnection connection;

        Ссылок: 1
        public static MySqlConnection GetConnection()
        {
            if (connection == null)
                connection = new MySqlConnection(connectionString);

            if (connection.State != System.Data.ConnectionState.Open)
                connection.Open();

            return connection;
        }

        Ссылок: 11
        public static MySqlCommand GetCommand(string sql)
        {
            return new MySqlCommand(sql, GetConnection());
        }
    }
}
```

Рисунок 26 – Код класса для взаимодействий с базой данных

На форме авторизации реализована кнопка «Войти», которая проверяет соответствие пароля и логина с существующей информацией в базе данных, а

также переносит пользователя на следующую форму выводя приветственное сообщение. Код данной кнопки представлен на рисунках 17 и 18.

```
Ссылок: 1
private void btnLogin_Click(object sender, EventArgs e)
{
    string login = txtLogin.Text.Trim();
    string password = txtPassword.Text.Trim();

    string sqlCheck = "SELECT COUNT(*) FROM users WHERE login = @login AND password_hash = @password";
    try
    {
        using (MySqlCommand cmdCheck = DbClass.Database.GetCommand(sqlCheck))
        {
            cmdCheck.Parameters.AddWithValue("@login", login);
            cmdCheck.Parameters.AddWithValue("@password", password);
            int count = Convert.ToInt32(cmdCheck.ExecuteScalar());

            if (count > 0)
            {
                string sqlUser = "SELECT id, full_name, role FROM users WHERE login = @login AND password_hash = @password";
                using (MySqlCommand cmdUser = DbClass.Database.GetCommand(sqlUser))
                {
                    cmdUser.Parameters.AddWithValue("@login", login);
                    cmdUser.Parameters.AddWithValue("@password", password);
                    using (var reader = cmdUser.ExecuteReader())
                    {
                        if (reader.Read())
                        {
                            CurrentUser.Id = reader.GetInt32("id");
                            CurrentUser.Name = reader.GetString("full_name");
                            CurrentUser.Role = reader.GetString("role");
                        }
                    }
                }
```

Рисунок 17 – Фрагмент кода для кнопки «Войти»

```
                string roleMsg = CurrentUser.Role == "client" ? "клиент" : "тренер";
                MessageBox.Show($"Добро пожаловать, {CurrentUser.Name}! Вы вошли как {roleMsg}.");
                this.Hide();
                new frmSessionsSchedule().Show();
                return;
            }
        }
        MessageBox.Show("Неверный логин или пароль");
    }
    catch (Exception ex) { MessageBox.Show("Ошибка: " + ex.Message); }
}
```

Рисунок 18 – Фрагмент кода для кнопки «Войти»

На форме расписания тренировок присутствует кнопка «Добавить» которая переносит на форму редактирования тренировок. Код для кнопки «Добавить» представлен на рисунке 19.

```
Ссылок: 1
private void btnAddSession_Click(object sender, EventArgs e)
{
    frmSessionEdit form = new frmSessionEdit();
    if (form.ShowDialog() == DialogResult.OK)
        LoadSessions();
    form.Dispose();
}
```

Рисунок 19 – Код для кнопки «Добавить» на форме расписания тренировок

На форме редактирования тренировок имеется combobox для заполнения тренера автоматически, предлагая выбрать желаемого. Код для загрузки данных тренера из базы данных представлен на рисунке 20.

```
Ссылок: 1
private void LoadTrainers()
{
    string sql = "SELECT id, full_name FROM users WHERE role='trainer'";
    LoadComboBox(sql, cmbTrainer, "full_name");
}
```

Рисунок 20 – Код для заполнения combobox тренеров

Из формы редактирования тренировок нажав на кнопку «Достижения» можно перейти на следующую форму с одноименным названием. На данной форме реализована кнопка «Добавить» которая переносит тренера на форму добавления достижений. Код для кнопки «Добавить» представлен на рисунке 21.

```
Ссылок: 1
private void btnAdd_Click(object sender, EventArgs e)
{
    frmAchievementEdit form = new frmAchievementEdit();
    form.ShowDialog();
    LoadAchievements();
    form.Dispose();
}
```

Рисунок 21 – Код для кнопки «Добавить» на форме достижений

Перейдя на форму добавления достижений можно заметить кнопку «Сохранить». При заполнении данных и нажатии на нее произведется добавление нового достижений на форму и сохранение информации в базу данных. Код для кнопки «Сохранить» представлен на рисунке 22.

```
Ссылок: 1
private void btnSave_Click(object sender, EventArgs e)
{
    if (string.IsNullOrWhiteSpace(txtTitle.Text) || dtDate.Value.Date > DateTime.Today)
    {
        MessageBox.Show("Проверьте название и дату!");
        return;
    }

    string sql = @"INSERT INTO achievements(client_id,sport_id,title,description,achievement_date,created_by)
                   VALUES(@c,@s,@t,@d,@ad,@cb)";
    try
    {
        using (MySqlCommand cmd = DbClass.Database.GetCommand(sql))
        {
            cmd.Parameters.AddWithValue("@c", CurrentUser.Id);
            cmd.Parameters.AddWithValue("@s", cmbSport.SelectedValue ?? (object)DBNull.Value);
            cmd.Parameters.AddWithValue("@t", txtTitle.Text.Trim());
            cmd.Parameters.AddWithValue("@d", txtDescription.Text.Trim() ?? (object)DBNull.Value);
            cmd.Parameters.AddWithValue("@ad", dtDate.Value.Date);
            cmd.Parameters.AddWithValue("@cb", CurrentUser.Id);
            cmd.ExecuteNonQuery();
        }
        MessageBox.Show("Добавлено!");
        DialogResult = DialogResult.OK;
        Close();
    }
    catch (Exception ex) { MessageBox.Show("Ошибка: " + ex.Message); }
}
```

Рисунок 22 – Код для кнопки «Сохранить» на форме добавления достижений

Полный код для всех форм и элементов представлен в Приложениях А-Ж.

2.3 Организация хранения и доступа к файлам посредством репозитория

В процессе разработки важно правильно организовать хранение его кода и базы данных. Для этого используются системы контроля версий [10]. На рисунках 23 и 24 представлены окно с введением команд для загрузки проекта и базы данных проекта в заранее созданный репозиторий.

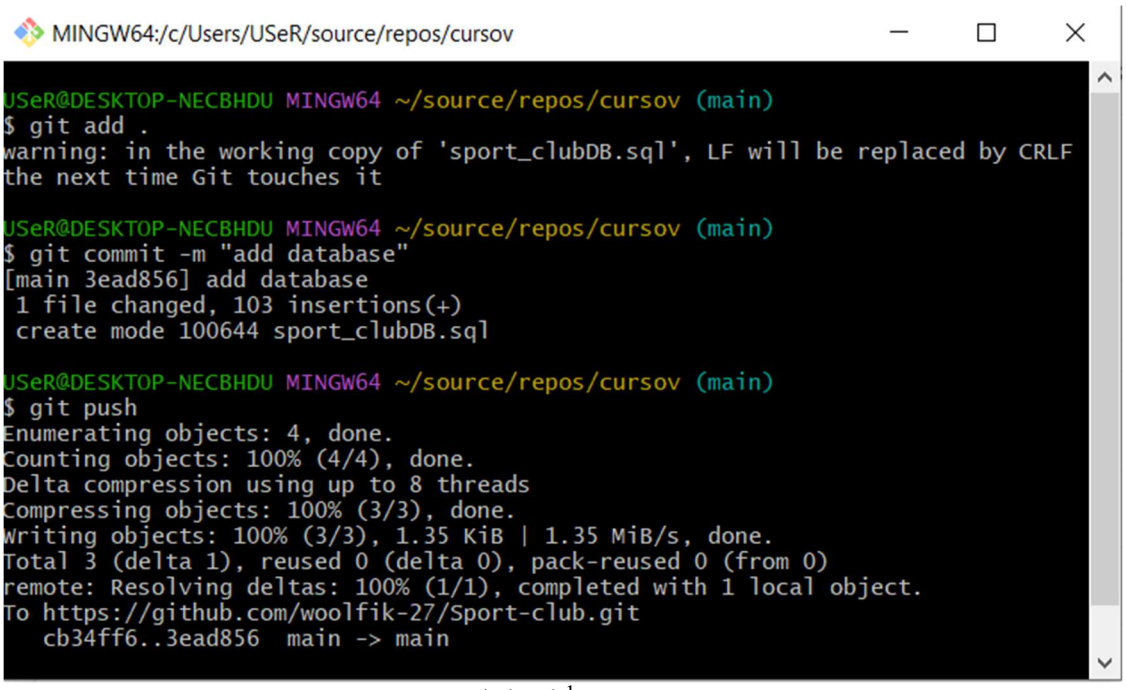

Рисунок 23 – Окно Git bash[1] с введенными командами

[1] Git Bash – это приложение для сред Microsoft Windows, эмулирующее работу командной строки Git. [Электронный ресурс] – URL: https://www.atlassian.com/ru/git/tutorials/git-bash Режим доступа: свободный (дата обращения: 22.04.2026).

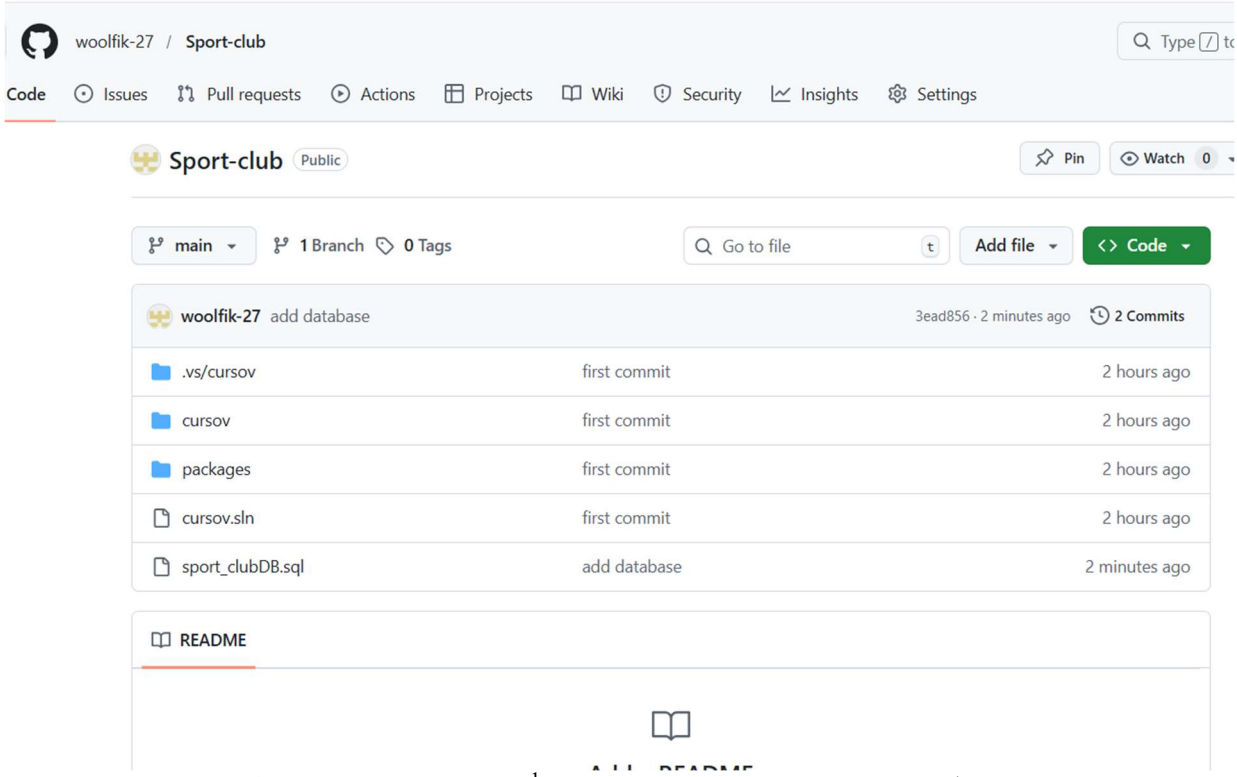

Рисунок 24 – репозиторий GIT[1] с загруженными проектом и базой данных

Использование репозитория помогает отслеживать изменения, вносить исправления и возвращаться к предыдущим версиям проекта. Это делает процесс разработки более удобным и безопасным [10]. Проект доступен на репозитории: https://github.com/woolfik-27/Sport-club .

2.4 Тестирование системы

Тестирование обязательная часть разработки, потому что позволяет выявить возможные ошибки и оценить стабильность работы. Тестирование проводилось на основе заранее подготовленных сценариев, которые показывают основные взаимодействия пользователя с системой [12].

В таблицах 6-10 приведены тестовые сценарии разработанного приложения.

[1] Git – это система для управления версиями исходного кода программ. [Электронный ресурс] – URL: https://skillbox.ru/media/code/chto_takoe_git_obyasnyaem_na_skhemakh/ Режим доступа: свободный (дата обращения: 22.04.2026).

Таблица 6 – Тестовый сценарий 1

Тестовый сценарий 1	ТС ПДТ 1
Приоритет тестирования	Средний
Заголовок/название теста	Проверка добавления тренировки
Краткое изложение теста	Проверка добавления тренировки
Этапы теста	1. Перейти на форму «Расписание тренировок». 2. Заполнить информацией поля «Тренер», «Спорт», «Статус», «Время начала тренировки», «Время конца тренировки», «Дата тренировки». 3. Нажать на кнопку «Сохранить» 4. Отследить сохранение результатов в базу данных
Тестовые данные	Петр Петров Фитнес Запланировано 10:00:00 11:00:00 19 марта 2026 г.
Ожидаемый результат	Сохранение информации тренировок в базе данных
Фактический результат	Информация сохраняется в базе данных
Статус	Зачет
Предварительное условие	Разработана форма «Редактирование» Добавлена кнопка «Сохранить»
Постусловие	Разработана форма «Редактирование» Добавлена кнопка «Сохранить»
Примечания/комментарии	Отсутствуют

Таблица 7 – Тестовый сценарий 2

Тестовый сценарий 1	ТС ППНФ 1
Приоритет тестирования	Высокий
Заголовок/название теста	Проверка перехода на форму «Расписание тренировок» по кнопке «Войти»
Краткое изложение теста	Проверка перехода на форму «Расписание тренировок» по кнопке «Войти»
Этапы теста	1. Перейти на страницу «Авторизация» 2. Заполнить информацией поля «Логин» и «Пароль» 3. Нажать на кнопку «Войти» 4. Отследить переход на форму «Расписание тренировок»
Тестовые данные	trainer1 123
Ожидаемый результат	Переход на форму «Расписание тренировок» работает корректно
Фактический результат	Переход на форму «Расписание тренировок» работает корректно
Статус	Зачет
Предварительное условие	Разработана форма «Авторизация» Разработана форма «Расписание тренировок» Добавлена кнопка «Войти» для перехода
Постусловие	Разработана форма «Авторизация» Разработана форма «Расписание тренировок» Добавлена кнопка «Войти» для перехода
Примечания/комментарии	Отсутствуют

Таблица 8 – Тестовый сценарий 3

Тестовый сценарий 1	ТС ПДТ 2
Приоритет тестирования	Средний
Заголовок/название теста	Проверка добавления тренировки
Краткое изложение теста	Проверка добавления тренировки
Этапы теста	1. Перейти на страницу «Расписание тренировок» 2. Заполнить информацией поля «Тренер», «Спорт», «Статус», «Время конца тренировки», «Дата тренировки» 3. Нажать на кнопку «Сохранить» 4. Отследить сохранение результатов в базу данных
Тестовые данные	Петр Петров Бокс Запланировано 12:00:00 13:00:00 11.03.2026
Ожидаемый результат	Сохранение информации о тренировки
Фактический результат	Информация о тренировке сохраняется
Статус	Зачет
Предварительное условие	Разработана форма «Расписание тренировок» Разработана форма «Редактирование тренировок» Добавлена кнопка «Сохранить»
Постусловие	Разработана форма «Расписание тренировок» Разработана форма «Редактирование тренировок» Добавлена кнопка «Сохранить»
Примечания/комментарии	Отсутствуют

Таблица 9 – Тестовый сценарий 4

Тестовый сценарий 1	ТС ППНФ 2
Приоритет тестирования	Высокий
Заголовок/название теста	Проверка перехода на форму «Авторизация» по кнопке «Зарегистрироваться»
Краткое изложение теста	Проверка перехода на форму «Регистрация» по кнопке «Зарегистрироваться»
Этапы теста	1. Перейти на страницу «Регистрация» 2. Заполнить информацией поля «Роль», «Имя», «Логин», «Пароль» 3. Нажать на кнопку «Зарегистрироваться» 4. Отследить переход на форму «Авторизация»
Тестовые данные	Тренер trainer1 123
Ожидаемый результат	Переход на форму «Авторизация» работает корректно
Фактический результат	Переход на форму «Авторизация» работает корректно
Статус	Зачет
Предварительное условие	Разработана форма «Авторизация» Разработана форма «Регистрация» Добавлена кнопка «Зарегистрироваться»
Постусловие	Разработана форма «Авторизация» Разработана форма «Регистрация» Добавлена кнопка «Зарегистрироваться»
Примечания/комментарии	Отсутствуют

Таблица 10 – Тестовый сценарий 5

Тестовый сценарий 1	ТС ППНФ 3
Приоритет тестирования	Средний
Заголовок/название теста	Проверка перехода на форму «Расписание тренировок» по кнопке «Назад» на форме «Достижения»
Краткое изложение теста	Проверка перехода на форму «Расписание тренировок» по кнопке «Назад» на форме «Достижения»
Этапы теста	1. Перейти на страницу «Достижения» 2. Нажать на кнопку «Назад» 3. Отследить переход на форму «Расписание тренировок»
Тестовые данные	Отсутствуют
Ожидаемый результат	Переход на форму «Расписание тренировок» работает корректно
Фактический результат	Переход на форму «Расписание тренировок» работает корректно
Статус	Зачет
Предварительное условие	Разработана форма «Расписание тренировок» Разработана форма «Достижения» Добавлена кнопка «Назад»
Постусловие	Разработана форма «Расписание тренировок» Разработана форма «Достижения» Добавлена кнопка «Назад»
Примечания/комментарии	Отсутствуют

ЗАКЛЮЧЕНИЕ

В ходе выполнения проекта была разработана информационная система, предназначенная для учета тренировок и достижений спортивного клуба и проанализирована предметная область деятельности спортивных клубов. Это позволило определить основные задачи, которые решит разрабатываемая система. Было установлено что универсальные системы не всегда подходят для решения специализированных задач и оказываются сложными для использования.

На основе анализа была сформулированы требования к системе и разработана структура базы данных. Данная структура была реализована с использованием SQL, что обеспечило целостность данных и стабильную работу системы.

В рамках разработки приложения был реализован пользовательский интерфейс, удобно взаимодействующий с системой. Были созданы формы авторизации и регистрации пользователей, форма отображения расписания тренировок и формы добавления и редактировать данных, а также формы с возможностью просмотра и добавления достижений клуба.

В ходе работе реализованы основные функции системы: добавление, редактирование, удаление и просмотр данных. Проведенное тестирование показало, что система корректно выполняет все заявленные функции.

Поставленная цель – разработка приложения учета тренировок и достижений в спортивном клубе – достигнута. Все задачи успешно выполнены.

Внедрение подобных информационных систем помогает повысить эффективность работы спортивных клубов, упростить управление данными и улучшить взаимодействие между участниками процесса.

Варианты для расширения функционала приложения:

— Внедрение системы статистики и аналитики;

— Реализация функций поиска и фильтрации данных по различным параметрам;

— Добавление возможности экспорта данных в различные форматы;

— Интеграция платежной системы;

— Добавление истории посещения клиента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

Книги под фамилией автора:

1) Гаврилов, М. В. Информатика и информационные технологии : учебник для среднего профессионального образования / М. В. Гаврилов, В. А. Климов. — 6-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 319 с. — (Профессиональное образование). — ISBN 978-5-534-20333-2. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/583523 (дата обращения: 14.03.2026).

2) Грекул, В. И. Проектирование информационных систем : учебник и практикум для вузов / В. И. Грекул, Н. Л. Коровкина, Г. А. Левочкина. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 404 с. — (Высшее образование). — ISBN 978-5-534-19505-7. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/590554 (дата обращения: 06.04.2026).

3) Зыков, С. В. Программирование : учебник и практикум для вузов / С. В. Зыков. — 2-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 285 с. — (Высшее образование). — ISBN 978-5-534-16031-4. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/583644 (дата обращения: 15.04.2026).

4) Советов, Б. Я. Информационные технологии : учебник для среднего профессионального образования / Б. Я. Советов, В. В. Цехановский. — 8-е изд., перераб. и доп. — Москва : Издательство Юрайт, 2026. — 414 с. — (Профессиональное образование). — ISBN 978-5-534-20053-9. — Текст : электронный // Образовательная платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/583524 (дата обращения: 26.03.2026).

5) Стружкин, Н. П. Базы данных: проектирование : учебник для среднего профессионального образования / Н. П. Стружкин, В. В. Годин. — Москва : Издательство Юрайт, 2026. — 477 с. — (Профессиональное образование). — ISBN 978-5-534-11635-9. — Текст : электронный // Образовательная

платформа Юрайт [сайт]. — URL: https://urait.ru/bcode/587735 (дата обращения: 07.04.2026).

Электронные ресурсы:

6) CRM (от англ. «Custome Relationship Management») – это функциональный инструмент, который помогает выстраивать и поддерживать отношения с клиентами. [Электронный ресурс] – URL: https://crm.ru/blog/crm-chto-eto-takoe Режим доступа: свободный (дата обращения: 20.03.2026).

7) ER (Entity-Relationship, или диаграмма «сущность-связь») – это схема, которая показывает, какие данные есть в системе и как они связаны друг с другом. [Электронный ресурс] – URL: https://kurshub.ru/journal/blog/chto-takoe-er-diagramma/ Режим доступа: свободный (дата обращения: 05.04.2026).

8) Git – это система для управления версиями исходного кода программ. [Электронный ресурс] – URL: https://skillbox.ru/media/code/chto_takoe_git_obyasnyaem_na_skhemakh/ Режим доступа: свободный (дата обращения: 22.04.2026).

9) Git Bash – это приложение для сред Microsoft Windows, эмулирующее работу командной строки Git. [Электронный ресурс] – URL: https://www.atlassian.com/ru/git/tutorials/git-bash Режим доступа: свободный (дата обращения: 22.04.2026).

10) GitHub Docs. Документация по использованию системы контроля версий Git и платформы GitHub. / [Электронный ресурс] // GitHub: [сайт]. — URL: https://docs.github.com/ — Режим доступа: свободный (дата обращения: 19.04.2026).

11) IDE (от англ. Integrated Development Environment, «интегрированная среда разработки») — интегрированная среда разработки, набор ПО для создания кода. [Электронный ресурс] – URL: https://blog.skillfactory.ru/glossary/ide/ Режим доступа: свободный (дата обращения: 30.03.2026).

12) MDN Web Docs. Документация по веб-разработке и программированию. / [Электронный ресурс] // Mozilla: [сайт]. — URL:

https://developer.mozilla.org/ — Режим доступа: свободный (дата обращения: 21.04.2026).

13) Microsoft Learn. Документация по разработке на платформе .NET и языке C#. / [Электронный ресурс] // Microsoft: [сайт]. — URL: https://learn.microsoft.com/ — Режим доступа: свободный (дата обращения: 27.03.2026).

14) MySQL Documentation. Руководство по системе управления базами данных MySQL. / [Электронный ресурс] // Oracle: [сайт]. — URL: https://dev.mysql.com/doc/ — Режим доступа: свободный (дата обращения: 27.03.2026).

15) SQL Tutorial. Основы языка структурированных запросов SQL. / [Электронный ресурс] // W3Schools: [сайт]. — URL: https://www.w3schools.com/sql/ — Режим доступа: свободный (дата обращения: 06.04.2026).

16) SQL-инъекция – техника, при которой злоумышленник пользуется недостатками в коде приложения, который отвечает за построение динамических SQL-запросов. [Электронный ресурс] – URL: https://www.php.net/manual/ru/security.database.sql-injection.php Режим доступа: свободный (дата обращения: 06.04.2026).

17) Stack Overflow. Вопросы и ответы по программированию и разработке программного обеспечения. / [Электронный ресурс] // Stack Overflow: [сайт]. — URL: https://stackoverflow.com/ — Режим доступа: свободный (дата обращения: 13.04.2026).

18) Официальный сайт фирмы «1С». Программные продукты и решения для автоматизации бизнеса. / [Электронный ресурс] // 1С: [сайт]. — URL: https://1c.ru/ — Режим доступа: свободный (дата обращения: 19.03.2026).

ПРИЛОЖЕНИЯ

ПРИЛОЖЕНИЕ А

Программный код страницы «Авторизация»

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using MySql.Data.MySqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement.StartPanel;

namespace cursov
{
    public partial class Authorization : Form
    {
        public Authorization()
        {
            InitializeComponent();
        }

        private void btnLogin_Click(object sender, EventArgs e)
        {
            string login = txtLogin.Text.Trim();
            string password = txtPassword.Text.Trim();

            string sqlCheck = "SELECT COUNT(*) FROM users WHERE login = @login AND
password_hash = @password";
            try
            {
                using (MySqlCommand cmdCheck = DbClass.Database.GetCom-
mand(sqlCheck))
                {
                    cmdCheck.Parameters.AddWithValue("@login", login);
                    cmdCheck.Parameters.AddWithValue("@password", password);
                    int count = Convert.ToInt32(cmdCheck.ExecuteScalar());

                    if (count > 0)
                    {
                        string sqlUser = "SELECT id, full_name, role FROM users
WHERE login = @login AND password_hash = @password";
                        using (MySqlCommand cmdUser = DbClass.Database.GetCom-
mand(sqlUser))
                        {
                            cmdUser.Parameters.AddWithValue("@login", login);
                            cmdUser.Parameters.AddWithValue("@password", password);
                            using (var reader = cmdUser.ExecuteReader())
                            {
                                if (reader.Read())
                                {
                                    CurrentUser.Id = reader.GetInt32("id");
                                    CurrentUser.Name =
reader.GetString("full_name");
                                    CurrentUser.Role = reader.GetString("role");
                                }
                            }
                        }
```

```
                        string roleMsg = CurrentUser.Role == "client" ? "клиент" :
"тренер";
                        MessageBox.Show($"Добро пожаловать, {CurrentUser.Name}! Вы
вошли как {roleMsg}.");
                        this.Hide();
                        new frmSessionsSchedule().Show();
                        return;
                    }
                }
                MessageBox.Show("Неверный логин или пароль");
            }
            catch (Exception ex) { MessageBox.Show("Ошибка: " + ex.Message); }
        }

        private void btnOpenRegistration_Click(object sender, EventArgs e)
        {
            this.Hide();
            new Registration(this).Show();
        }
    }
}
```

ПРИЛОЖЕНИЕ Б

Программный код страницы «Регистрация»

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using MySql.Data.MySqlClient;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace cursov
{
    public partial class Registration : Form
    {
        private Authorization authorizationForm;

        public Registration(Authorization authorizationForm)
        {
            InitializeComponent();
            this.authorizationForm = authorizationForm;
        }

        private void Registration_Load(object sender, EventArgs e)
        {
            cmbRole.Items.Add("Клиент");
            cmbRole.Items.Add("Тренер");
            cmbRole.SelectedIndex = 0;
        }

        private void btnRegister_Click(object sender, EventArgs e)
        {
            string name = txtName.Text.Trim();
            string login = txtLogin.Text.Trim();
            string password = txtPassword.Text.Trim();

            if (name == "" || login == "" || password == "")
            {
                MessageBox.Show("Заполните все поля");
                return;
            }


            string role = cmbRole.SelectedIndex == 1 ? "trainer" : "client";

            try
            {

                string checkSql = "SELECT COUNT(*) FROM users WHERE login = @login";

                using (MySqlCommand cmd = DbClass.Database.GetCommand(checkSql))
                {
                    cmd.Parameters.AddWithValue("@login", login);

                    int count = Convert.ToInt32(cmd.ExecuteScalar());

                    if (count > 0)
                    {
                        MessageBox.Show("Логин уже занят");
                        return;
                    }
```

```
                }

                string insertSql = @"
                    INSERT INTO users (login, password_hash, full_name, role)
                    VALUES (@login, @password, @name, @role)
                ";

                using (MySqlCommand cmd = DbClass.Database.GetCommand(insertSql))
                {
                    cmd.Parameters.AddWithValue("@login", login);
                    cmd.Parameters.AddWithValue("@password", password);
                    cmd.Parameters.AddWithValue("@name", name);
                    cmd.Parameters.AddWithValue("@role", role);

                    cmd.ExecuteNonQuery();
                }

                MessageBox.Show("Регистрация успешна!");

                authorizationForm.Show();
                this.Close();
            }
            catch (Exception ex)
            {
                MessageBox.Show(ex.Message);
            }
        }

        private void btnBack_Click(object sender, EventArgs e)
        {
            authorizationForm.Show();
            this.Close();
        }
    }
}
```

ПРИЛОЖЕНИЕ В

Программный код страницы «Расписание тренировок»

```
using MySql.Data.MySqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
using static cursov.DbClass;

namespace cursov
{
    public partial class frmSessionsSchedule : Form
    {
        public frmSessionsSchedule()
        {
            InitializeComponent();
            this.FormClosing += new FormClosingEventHandler(frmSessionsSched-
ule_FormClosing);
        }

        private void frmSessionsSchedule_FormClosing(object sender, FormClosingEven-
tArgs e)
        {
            if (e.CloseReason == CloseReason.UserClosing)
            {
                Application.Exit();
            }
        }
        private void LoadSessions()
        {
            string sql;
            if (CurrentUser.Role == "client")
            {
                sql = @"
                    SELECT s.id, s.session_date AS 'Дата', s.start_time AS 'Начало',
                           s.end_time AS 'Конец', sp.name AS 'Спорт',
                           u.full_name AS 'Тренер',
                           CASE WHEN s.status = 'planned' THEN 'Запланирована'
                                WHEN s.status = 'completed' THEN 'Завершена'
                                WHEN s.status = 'canceled' THEN 'Отменена' END AS
'Статус'
                    FROM sessions s JOIN sports sp ON sp.id = s.sport_id
                    JOIN users u ON u.id = s.trainer_id WHERE s.client_id = @userId
                    ORDER BY s.session_date, s.start_time";
            }
            else
            {
                sql = @"
                    SELECT s.id, s.session_date AS 'Дата', s.start_time AS 'Начало',
                           s.end_time AS 'Конец', sp.name AS 'Спорт',
                           uc.full_name AS 'Клиент', ut.full_name AS 'Тренер',
                           CASE WHEN s.status = 'planned' THEN 'Запланирована'
                                WHEN s.status = 'completed' THEN 'Завершена'
                                WHEN s.status = 'canceled' THEN 'Отменена' END AS
'Статус'
                    FROM sessions s JOIN sports sp ON sp.id = s.sport_id
                    JOIN users uc ON uc.id = s.client_id JOIN users ut ON ut.id =
s.trainer_id
```

```
                ORDER BY s.session_date, s.start_time";
            }

            try
            {
                using (MySqlCommand cmd = DbClass.Database.GetCommand(sql))
                {
                    if (CurrentUser.Role == "client")
                        cmd.Parameters.AddWithValue("@userId", CurrentUser.Id);

                    using (MySqlDataAdapter adapter = new MySqlDataAdapter(cmd))
                    {
                        DataTable table = new DataTable();
                        adapter.Fill(table);
                        dgvSessions.DataSource = table;
                        if (dgvSessions.Columns.Contains("id"))
                            dgvSessions.Columns["id"].Visible = false;
                    }
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка загрузки расписания: " + ex.Message);
            }
        }

        private void frmSessionsSchedule_Load(object sender, EventArgs e)
        {
            // ☆ Кнопки по ролям
            if (CurrentUser.Role == "client")
            {
                btnEdit.Visible = false;
            }
            else
            {
                btnEdit.Visible = true;
            }

            LoadSessions();
        }

        private void btnAddSession_Click(object sender, EventArgs e)
        {
            frmSessionEdit form = new frmSessionEdit();
            if (form.ShowDialog() == DialogResult.OK)
                LoadSessions();
            form.Dispose();
        }

        private void btnEdit_Click(object sender, EventArgs e)
        {
            if (dgvSessions.CurrentRow == null || dgvSessions.Curren-
tRow.Cells["id"].Value == null)
            {
                MessageBox.Show("Выберите тренировку для редактирования!");
                return;
            }

            int id = Convert.ToInt32(dgvSessions.CurrentRow.Cells["id"].Value);
            frmSessionEdit form = new frmSessionEdit(id);
            if (form.ShowDialog() == DialogResult.OK)
                LoadSessions();
            form.Dispose();
        }
```

```
        private void btnAchievements_Click(object sender, EventArgs e)
        {
            this.Hide();

            Achievements form = new Achievements();
            form.ShowDialog();

            this.Show();
        }
    }
}
```

ПРИЛОЖЕНИЕ Г

Программный код страницы «Редактирование тренировок»

```
using MySql.Data.MySqlClient;
using MySqlX.XDevAPI;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace cursov
{
    public partial class frmSessionEdit : Form
    {
        private int sessionId = -1;

        public frmSessionEdit()
        {
            InitializeComponent();
        }

        public frmSessionEdit(int id)
        {
            InitializeComponent();
            sessionId = id;
        }

        private void frmSessionEdit_Load(object sender, EventArgs e)
        {
            LoadClients();
            LoadTrainers();
            LoadSports();

            cmbStatus.Items.Clear();
            cmbStatus.Items.Add("Запланирована");
            cmbStatus.Items.Add("Завершена");
            cmbStatus.Items.Add("Отменена");
            cmbStatus.SelectedIndex = 0;

            if (sessionId != -1)
                LoadSessionData();
        }

        private void LoadClients()
        {
            string sql = "SELECT id, full_name FROM users WHERE role='client'";
            LoadComboBox(sql, cmbClient, "full_name");
        }

        private void LoadTrainers()
        {
            string sql = "SELECT id, full_name FROM users WHERE role='trainer'";
            LoadComboBox(sql, cmbTrainer, "full_name");
        }

        private void LoadSports()
        {
            string sql = "SELECT id, name FROM sports";
            LoadComboBox(sql, cmbSport, "name");
```

```
        }

        private void LoadComboBox(string sql, ComboBox cmb, string displayMember)
        {
            try
            {
                using (MySqlCommand cmd = DbClass.Database.GetCommand(sql))
                using (MySqlDataAdapter adapter = new MySqlDataAdapter(cmd))
                {
                    DataTable dt = new DataTable();
                    adapter.Fill(dt);
                    cmb.DataSource = dt;
                    cmb.DisplayMember = displayMember;
                    cmb.ValueMember = "id";
                }
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка загрузки списка: " + ex.Message);
            }
        }
        private void LoadSessionData()
        {
            string sql = @"
                SELECT s.*, sp.name as sport_name, uc.full_name as client_name,
                       ut.full_name as trainer_name
                FROM sessions s
                JOIN sports sp ON sp.id = s.sport_id
                JOIN users uc ON uc.id = s.client_id
                JOIN users ut ON ut.id = s.trainer_id
                WHERE s.id = @id";

            try
            {
                using (MySqlCommand cmd = DbClass.Database.GetCommand(sql))
                {
                    cmd.Parameters.AddWithValue("@id", sessionId);
                    using (var reader = cmd.ExecuteReader())
                    {
                        if (reader.Read())
                        {
                            cmbClient.SelectedValue = reader["client_id"];
                            cmbTrainer.SelectedValue = reader["trainer_id"];
                            cmbSport.SelectedValue = reader["sport_id"];

                            dtDate.Value = reader.GetDateTime("session_date");
                            dtStart.Value = DateTime.Today + reader.Get-
TimeSpan("start_time");
                            dtEnd.Value = DateTime.Today + reader.Get-
TimeSpan("end_time");

                            string statusDb = reader["status"].ToString();
                            switch (statusDb)
                            {
                                case "planned": cmbStatus.SelectedItem =
"Запланирована"; break;
                                case "completed": cmbStatus.SelectedItem =
"Завершена"; break;
                                case "canceled": cmbStatus.SelectedItem =
"Отменена"; break;
                                default: cmbStatus.SelectedIndex = 0; break;
                            }
                        }
                    }
                }
```

```
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка загрузки данных тренировки: " + ex.Message);
            }
        }

        private void btnSave_Click(object sender, EventArgs e)
        {
            TimeSpan startTime = dtStart.Value.TimeOfDay;
            TimeSpan endTime = dtEnd.Value.TimeOfDay;

            if (endTime <= startTime)
            {
                MessageBox.Show("Время окончания должно быть больше начала!");
                return;
            }
            if (cmbClient.SelectedValue == null || cmbTrainer.SelectedValue == null
|| cmbSport.SelectedValue == null)
            {
                MessageBox.Show("Выберите клиента, тренера и спорт!");
                return;
            }

            string sql;
            if (sessionId == -1)
            {
                sql = @"
                    INSERT INTO sessions (trainer_id, client_id, sport_id, ses-
sion_date,
                    start_time, end_time, status, created_by)
                    VALUES (@trainer, @client, @sport, @date, @start, @end, @status,
@creator)";
            }
            else
            {
                sql = @"
                    UPDATE sessions SET trainer_id=@trainer, client_id=@client,
sport_id=@sport,
                    session_date=@date, start_time=@start, end_time=@end, sta-
tus=@status
                    WHERE id=@id";
            }

            try
            {
                using (MySqlCommand cmd = DbClass.Database.GetCommand(sql))
                {
                    cmd.Parameters.AddWithValue("@trainer", cmbTrainer.Select-
edValue);
                    cmd.Parameters.AddWithValue("@client", cmbClient.SelectedValue);
                    cmd.Parameters.AddWithValue("@sport", cmbSport.SelectedValue);
                    cmd.Parameters.AddWithValue("@date", dtDate.Value.Date);
                    cmd.Parameters.AddWithValue("@start", startTime);
                    cmd.Parameters.AddWithValue("@end", endTime);

                    string statusDb = cmbStatus.SelectedItem.ToString();
                    switch (statusDb)
                    {
                        case "Запланирована": cmd.Parameters.AddWithValue("@status",
"planned"); break;
                        case "Завершена": cmd.Parameters.AddWithValue("@status",
"completed"); break;
                        case "Отменена": cmd.Parameters.AddWithValue("@status",
"canceled"); break;
```

```
                        default: cmd.Parameters.AddWithValue("@status", "planned");
break;
                    }

                    if (sessionId == -1)
                        cmd.Parameters.AddWithValue("@creator", CurrentUser.Id);
                    else
                        cmd.Parameters.AddWithValue("@id", sessionId);

                    cmd.ExecuteNonQuery();
                }
                MessageBox.Show("Тренировка сохранена!");
                this.DialogResult = DialogResult.OK;
                this.Close();
            }
            catch (MySqlException ex) when (ex.Number == 1062)
            {
                MessageBox.Show("Тренер уже занят в это время!");
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка сохранения: " + ex.Message);
            }
        }
    }
}
```

ПРИЛОЖЕНИЕ Д

Программный код страницы «Достижения»

```
using MySql.Data.MySqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace cursov
{
    public partial class Achievements : Form
    {
        public Achievements()
        {
            InitializeComponent();
        }

        private void Achievements_Load(object sender, EventArgs e)
        {
            btnAdd.Visible = CurrentUser.Role == "trainer";
            LoadAchievements();
        }

        private void LoadAchievements()
        {
            string sql = @"
                SELECT a.id, a.title AS 'Название', a.description AS 'Описание',
                       a.achievement_date AS 'Дата',
                       CASE WHEN a.client_id = @userId THEN 'Моё' ELSE 'Общее' END
AS 'Тип'
                FROM achievements a
                WHERE 1=1  -- Всегда что-то покажет
                ORDER BY a.achievement_date DESC
                LIMIT 10";  // Ограничение для теста

            try
            {
                using (MySqlCommand cmd = DbClass.Database.GetCommand(sql))
                {
                    cmd.Parameters.AddWithValue("@userId", CurrentUser.Id);
                    using (MySqlDataAdapter adapter = new MySqlDataAdapter(cmd))
                    {
                        DataTable table = new DataTable();
                        adapter.Fill(table);

                        if (table.Rows.Count == 0)
                        {
                            table.Rows.Add(0, "Нет достижений", "Добавьте первое че-
рез кнопку", DateTime.Now.Date, "Инфо");
                            MessageBox.Show($"Пустая таблица achievements.\nCurren-
tUser.Id = {CurrentUser.Id}\nДобавьте тестовые данные!");
                        }

                        dgvAchievements.DataSource = table;
                        if (dgvAchievements.Columns.Contains("id"))
                            dgvAchievements.Columns["id"].Visible = false;
                    }
                }
```

```
            }
            catch (Exception ex)
            {
                MessageBox.Show("Ошибка БД: " + ex.Message + "\nID=" + Curren-
tUser.Id);
            }
        }

        private void btnAdd_Click(object sender, EventArgs e)
        {
            frmAchievementEdit form = new frmAchievementEdit();
            form.ShowDialog();
            LoadAchievements();
            form.Dispose();
        }

        private void btnBackToTrain_Click(object sender, EventArgs e)
        {
            this.DialogResult = DialogResult.OK;
            this.Close();
        }
    }
}
```

ПРИЛОЖЕНИЕ Е

Программный код страницы «Редактирование достижений»

```
using MySql.Data.MySqlClient;
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace cursov
{
    public partial class frmAchievementEdit : Form
    {
        public frmAchievementEdit()
        {
            InitializeComponent();
        }

        private void frmAchievementEdit_Load(object sender, EventArgs e)
        {
            LoadSports();
        }

        private void LoadSports()
        {
            try
            {
                using (MySqlCommand cmd = DbClass.Database.GetCommand("SELECT
id,name FROM sports"))
                using (MySqlDataAdapter adapter = new MySqlDataAdapter(cmd))
                {
                    var dt = new System.Data.DataTable();
                    adapter.Fill(dt);
                    cmbSport.DataSource = dt;
                    cmbSport.DisplayMember = "name";
                    cmbSport.ValueMember = "id";
                }
            }
            catch (Exception ex) { MessageBox.Show("Ошибка: " + ex.Message); }
        }

        private void btnSave_Click(object sender, EventArgs e)
        {
            if (string.IsNullOrWhiteSpace(txtTitle.Text) || dtDate.Value.Date >
DateTime.Today)
            {
                MessageBox.Show("Проверьте название и дату!");
                return;
            }

            string sql = @"INSERT INTO achievements(client_id,sport_id,title,de-
scription,achievement_date,created_by)
                           VALUES(@c,@s,@t,@d,@ad,@cb)";
            try
            {
                using (MySqlCommand cmd = DbClass.Database.GetCommand(sql))
                {
                    cmd.Parameters.AddWithValue("@c", CurrentUser.Id);
```

```
                    cmd.Parameters.AddWithValue("@s", cmbSport.SelectedValue ?? (ob-
ject)DBNull.Value);
                    cmd.Parameters.AddWithValue("@t", txtTitle.Text.Trim());
                    cmd.Parameters.AddWithValue("@d", txtDescription.Text.Trim() ??
(object)DBNull.Value);
                    cmd.Parameters.AddWithValue("@ad", dtDate.Value.Date);
                    cmd.Parameters.AddWithValue("@cb", CurrentUser.Id);
                    cmd.ExecuteNonQuery();
                }
                MessageBox.Show("Добавлено!");
                DialogResult = DialogResult.OK;
                Close();
            }
            catch (Exception ex) { MessageBox.Show("Ошибка: " + ex.Message); }
        }
    }
}
```

ПРИЛОЖЕНИЕ Ж

Скрипт базы данных проекта

```
CREATE DATABASE IF NOT EXISTS sport_club;
USE sport_club;

CREATE TABLE users (
   id INT AUTO_INCREMENT PRIMARY KEY,
   login VARCHAR(50) NOT NULL UNIQUE,
   password_hash VARCHAR(255) NOT NULL,
   full_name VARCHAR(100) NOT NULL,
   role ENUM('client','trainer') NOT NULL,
   created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE sports (
   id INT AUTO_INCREMENT PRIMARY KEY,
   name VARCHAR(50) NOT NULL UNIQUE
) ENGINE=InnoDB;

CREATE TABLE sessions (
   id INT AUTO_INCREMENT PRIMARY KEY,

   trainer_id INT NOT NULL,
   client_id INT NOT NULL,
   sport_id INT NOT NULL,

   session_date DATE NOT NULL,
   start_time TIME NOT NULL,
   end_time TIME NOT NULL,

   status ENUM('planned','completed','canceled')
      NOT NULL DEFAULT 'planned',

   created_by INT NOT NULL,
   created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

   CONSTRAINT chk_time CHECK (start_time < end_time),

   FOREIGN KEY (trainer_id) REFERENCES users(id) ON DELETE CASCADE,
   FOREIGN KEY (client_id) REFERENCES users(id) ON DELETE CASCADE,
   FOREIGN KEY (sport_id) REFERENCES sports(id) ON DELETE RESTRICT,
   FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE RE-
STRICT
) ENGINE=InnoDB;
```

```
CREATE UNIQUE INDEX ux_trainer_datetime
ON sessions (trainer_id, session_date, start_time);

CREATE INDEX ix_sessions_client ON sessions (client_id);
CREATE INDEX ix_sessions_date ON sessions (session_date);

CREATE TABLE achievements (
   id INT AUTO_INCREMENT PRIMARY KEY,

   title VARCHAR(100) NOT NULL,
   description TEXT,
   achievement_date DATE NOT NULL,

   sport_id INT NULL,

   created_by INT NOT NULL,
   created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

   FOREIGN KEY (sport_id) REFERENCES sports(id) ON DELETE SET NULL,
   FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE RE-
STRICT
) ENGINE=InnoDB;


CREATE INDEX ix_achievements_date
ON achievements (achievement_date);


INSERT INTO users (login, password_hash, full_name, role)
VALUES
('client1', '123', 'Иван Иванов', 'client'),
('client2', '123', 'Алексей Смирнов', 'client'),
('trainer1', '123', 'Петр Петров', 'trainer');

INSERT INTO sports (name)
VALUES
('Фитнес'),
('Бокс'),
('Йога');

INSERT INTO sessions
(trainer_id, client_id, sport_id, session_date, start_time, end_time, status, cre-
ated_by)
```

```
VALUES
(3, 1, 1, '2026-03-10', '10:00', '11:00', 'planned', 3),
(3, 2, 2, '2026-03-11', '12:00', '13:00', 'completed', 3),
(3, 1, 3, '2026-03-12', '14:00', '15:00', 'canceled', 3);

INSERT INTO achievements
(title, description, achievement_date, sport_id, created_by)
VALUES
('Лучший спортивный клуб года',
 'Победа в городском конкурсе среди фитнес-клубов',
 '2025-12-20', NULL, 3),

('Победа в турнире по боксу',
 'Первое место в региональном соревновании',
 '2025-11-10', 2, 3),

('Организация фестиваля йоги',
 'Проведение городского мероприятия',
 '2025-10-05', 3, 3);
```